

Python
for DA

Summary Session 5



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

- 

Камера должна быть включена на протяжении всего занятия
- 

В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
- 

Вести себя уважительно и этично по отношению к остальным участникам занятия
- 

Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
- 

Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя



ПОВТОРЕНИЕ ИЗУЧЕННОГО



Загрузка датасета [data_for_cleaning](#)

```
import numpy as np
import pandas as pd

df = pd.read_csv('.\data_for_cleaning.csv')
print(df.head(5))
print(df.tail(5))
print(df.info())
print(df.describe())
```

Стать IT-шником может каждый



Определение отсутствующих значений



Проблема качества данных: пропущенные значения

В pandas для обнаружения отсутствующих значений в DataFrame используются методы `isnull()` и `notnull()`

Экспресс-опрос

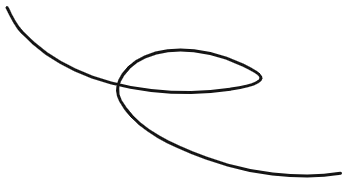
?

Что такое isnull()?



isnull() —

это метод, который возвращает булевый DataFrame той же формы, что и исходный DataFrame, в котором каждая ячейка имеет значение True, если соответствующая ячейка в исходном DataFrame имеет значение NaN (Not a Number) или None, и False в противном случае.



Метод isnull()



- Синтаксис: `DataFrame.isnull()`
- Возвращает: `DataFrame` с булевыми значениями (True для пропущенных значений и False для непропущенных)
- Параметры: Этот метод не принимает никаких параметров

Пример метода isnull()

```
missing_values = df.isnull()  
print(missing_values)
```

Экспресс-опрос

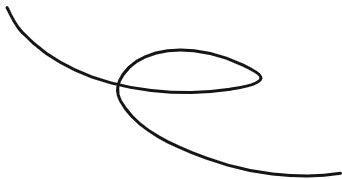
?

Для чего используется notnull()?



notnull() —

это метод, обратный методу `isnull()`. Возвращает булевый `DataFrame` той же формы, что и исходный `DataFrame`, где каждая ячейка имеет значение `True`, если соответствующая ячейка в исходном `DataFrame` не `NaN` (Not a Number) или `None`, и `False` в противном случае.



Метод notnull()



- Синтаксис: `DataFrame.notnull()`
- Возвращает: `DataFrame` с булевыми значениями (True для не пропущенных значений и False для пропущенных значений)
- Параметры: Этот метод не принимает никаких параметров

Пример метода notnull()

```
non_missing_values = df.notnull()  
print(non_missing_values)
```

Использование датасета

```
missing_values = df.isnull()

non_missing_values = df.notnull()

print("Пропущенные значения:")
print(missing_values)

print("\nНепропущенные значения:")
print(non_missing_values)
```

Стать IT-шником может каждый



Обработка пропущенных значений

Часть процесса очистки данных: обработка пропущенных значений

В pandas для обработки пропущенных значений используются методы `fillna()` и `dropna()`

Экспресс-опрос

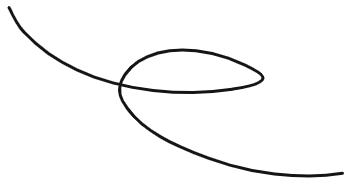
?

Для чего используется fillna()?



fillna() —

это метод, который используется для заполнения отсутствующих значений в DataFrame указанными значениями. Позволяет заменить NaN (Not a Number) или отсутствующие значения на указанное скалярное значение, словарь или использовать различные методы, такие как 'ffill' (прямое заполнение) или 'bfill' (обратное заполнение).



Метод fillna()



- Синтаксис: `DataFrame.fillna(value=None, method=None, axis=None, inplace=False, limit=None, downcast=None)`
- Возвращает: `DataFrame` с заполненными или замененными недостающими значениями

Параметры метода fillna()

Параметр	Пояснение
value	Скаляр, dict, Series или DataFrame. Значение, используемое для заполнения недостающих значений
метод	{'backfill', 'bfill', 'pad', 'ffill', None}, по умолчанию None. Метод, используемый для заполнения пропущенных значений
axis	{0 или 'index', 1 или 'columns'}. Ось, вдоль которой заполняются пропущенные значения
inplace	bool, по умолчанию False. Модифицировать DataFrame на месте (не создавать новый объект)
limit	int, по умолчанию None. Максимальное количество последовательных NaN для заполнения
downcast	dict, по умолчанию None. Словарь item→dtype с тем, что нужно передать вниз, если это возможно

Пример метода fillna()

```
df['Age'].fillna(df['Age'].mean(), inplace=True)
```

```
df['Gender'].fillna('Unknown', inplace=True)
```

```
df['Department'].fillna(method='ffill', inplace=True)
```

Экспресс-опрос

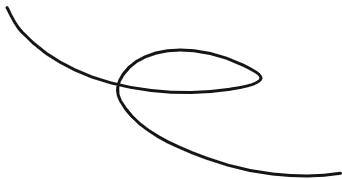
?

Для чего используется dropna()?



dropna() —

это метод, который используется для удаления отсутствующих значений из DataFrame. Позволяет удалять строки или столбцы, в которых любой или все элементы являются NaN (Not a Number).



Метод dropna()



- Синтаксис: `DataFrame.dropna(axis=0, how='any', thresh=None, subset=None, inplace=False)`
- Возвращает: `DataFrame` с удаленными недостающими значениями

Параметры метода `dropna()`

Параметр	Пояснение
<code>axis</code>	{0 или 'index', 1 или 'columns'}. Ось, по которой нужно отбросить пропущенные значения
<code>how</code>	{'any', 'all'}, по умолчанию 'any'. Определяет, удалять ли строку или столбец из DataFrame, где любой или все элементы являются NaN
<code>thresh</code>	int, по умолчанию None. Требуется, чтобы количество значений, не являющихся NaN, было максимальным
<code>subset</code>	array-like, по умолчанию None. Метки по другим осям, которые необходимо учитывать, например, если вы удаляете строки, то это будет список столбцов, которые необходимо включить
<code>inplace</code>	bool, по умолчанию False. Модифицировать DataFrame на месте (не создавать новый объект)

Пример метода dropna()

Удаляем строки, в которых отсутствует какой-либо элемент в DataFrame

```
df.dropna(inplace=True)
```

Удаление строк, в которых все элементы отсутствуют в DataFrame

```
df.dropna(how='all', inplace=True)
```

Отбрасываем строки, в которых столбец 'Зарплата' или 'Пол' имеет недостающие значения

```
df.dropna(subset=['Salary', 'Gender'], inplace=True)
```

Использование датасета

```
df['Age'].fillna(df['Age'].mean(), inplace=True)
df['Gender'].fillna('Unknown', inplace=True)
df['Department'].fillna(method='ffill', inplace=True)
df.dropna(inplace=True)
print(df)
```

Стать IT-шником может каждый



Подстановка недостающих значений

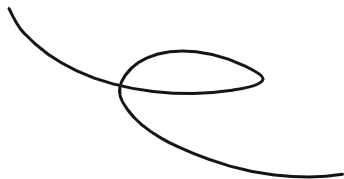
Подход к обработке отсутствующих данных: подстановка недостающих значений

Pandas предоставляет различные методы для подстановки отсутствующих значений с использованием таких статистических показателей, как среднее, медиана и мода



Использование среднего значения —

это метод, который используется для подстановки пропущенных значений. Можно использовать среднее значение для замены пропущенных значений в числовых столбцах.



Использование среднего значения



- Синтаксис: `DataFrame['имя_столбца'].fillna(value=df['имя_столбца'].mean(), inplace=True)`
- Возвращает: DataFrame с отсутствующими значениями в указанном столбце, замененными средним значением

Пример использования среднего значения

```
df['Age'].fillna(value=df['Age'].mean(), inplace=True)
```

Экспресс-опрос

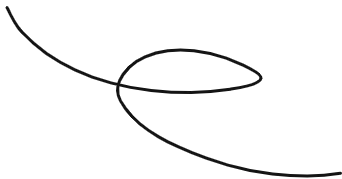
?

Что такое медиана?



Медиана —

это мера, которая представляет собой срединное значение в наборе данных. Можно использовать медиану для замены пропущенных значений в числовых столбцах, особенно если данные содержат провалы.



Использование медианы



- Синтаксис: `DataFrame['имя_столбца'].fillna(value=df['имя_столбца'].median(), inplace=True)`
- Возвращает: DataFrame с отсутствующими значениями в указанном столбце, замененными медианным значением

Пример использования медианы

```
df['Salary'].fillna(value=df['Salary'].median(), inplace=True)
```

Экспресс-опрос

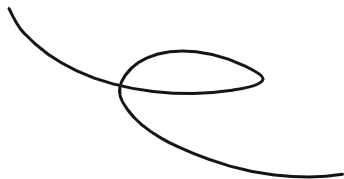


Что такое мода?



Мода —

это мера, которая представляет собой наиболее часто встречающееся значение в наборе данных. Можно использовать для замены отсутствующих значений в категориальных столбцах.



Использование моды



- Синтаксис: `DataFrame['имя_столбца'].fillna(value=df['имя_столбца'].mode()[0], inplace=True)`
- Возвращает: DataFrame с отсутствующими значениями в указанном столбце, замененными значением режима

Пример использования моды

```
df['Gender'].fillna(value=df['Gender'].mode()[0], inplace=True)
```

Использование датасета

```
df['Age'].fillna(value=df['Age'].mean(), inplace=True)
df['Salary'].fillna(value=df['Salary'].median(), inplace=True)
df['Gender'].fillna(value=df['Gender'].mode()[0], inplace=True)
print(df)
```




Удаление дубликатов

Проблема искажения результатов анализа: дублирующиеся строки

Pandas предоставляет метод `drop_duplicates()`, чтобы легко удалить дубликаты строк из DataFrame

Экспресс-опрос

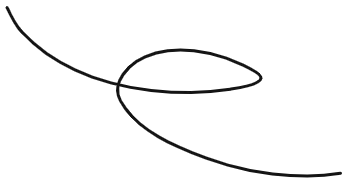
?

Что делает метод `drop_duplicates` и какие у него параметры?



drop_duplicates() —

это метод, который используется для удаления дубликатов строк из DataFrame на основе указанных столбцов или всех столбцов.



Метод drop_duplicates()

- Синтаксис: `DataFrame.drop_duplicates(subset=None, keep='first', inplace=False, ignore_index=False)`
- Возвращает: `DataFrame` с удаленными дублирующимися строками

Параметры метода `drop_duplicates()`

Параметр	Пояснение
<code>subset</code>	Метка столбца или последовательность меток, необязательно. Подмножество столбцов для выявления дубликатов. По умолчанию учитываются все столбцы
<code>keep</code>	{'first', 'last', False}, по умолчанию 'first'. Определяет, какой дубликат сохранять. 'first' сохраняет первое вхождение, 'last' сохраняет последнее вхождение, а False удаляет все дубликаты
<code>inplace</code>	bool, по умолчанию False. Модифицировать DataFrame на месте (не создавать новый объект)
<code>ignore_index</code>	bool, по умолчанию False. Если значение True, то результирующий DataFrame будет иметь новый RangeIndex

Пример использования метода drop_duplicates()

```
df.drop_duplicates(inplace=True)  
df.drop_duplicates(subset=['Name', 'Age'], inplace=True)
```

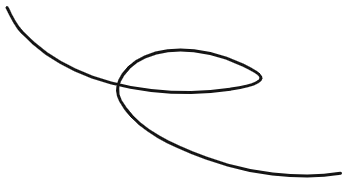
Использование датасета

```
print("Изначальный DataFrame:")  
print(df)  
  
df.drop_duplicates(inplace=True)  
  
print("\nDataFrame после удаления дубликатов:")  
print(df)
```



Переименование столбцов —

это распространенная операция при работе с pandas DataFrames, для того, чтобы сделать имена столбцов более описательными или исправить опечатки или несоответствия в именах столбцов.



Экспресс-опрос

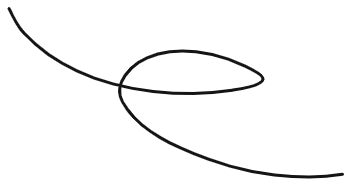
?

Для чего используется rename()?



rename() —

это метод, который позволяет переименовывать столбцы в DataFrame, указывая словарь, который сопоставляет текущие имена столбцов с новыми именами столбцов.



Метод rename()

- Синтаксис: `DataFrame.rename(columns={'текущее_имя': 'новое_имя'}, inplace=False)`
- Возвращает: `DataFrame` с переименованными столбцами

Параметры метода rename()

Параметр	Пояснение
columns	Диктоподобный объект или функция, необязательно. Словарь или функция для переименования меток осей
inplace	bool, по умолчанию False. Модифицировать DataFrame на месте (не создавать новый объект)

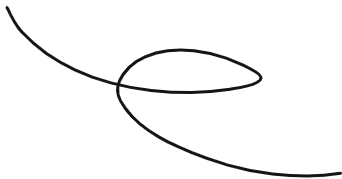
Использование датасета

```
print("Изначальный DataFrame:")  
print(df)  
  
df.rename(columns={'Joining_Date': 'JoinDate'}, inplace=True)  
  
print("\nDataFrame после переименования поля:")  
print(df)
```



Изменение типов данных —

это распространенная операция при работе с pandas DataFrames, особенно для преобразования столбцов в соответствующие типы данных для целей анализа или визуализации.



Экспресс-опрос

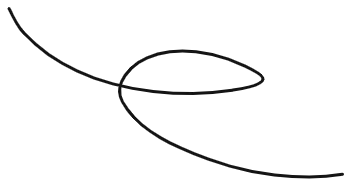
?

Какой метод используется для изменения типов столбцов в DataFrame?



astype() —

это метод, который используется для изменения типов данных столбцов в DataFrame на указанный тип данных.



Метод astype()

- Синтаксис: `DataFrame.astype(dtype, copy=True, errors='raise')`
- Возвращает: `DataFrame` с измененными типами данных



Параметры метода astype()

Параметр	Пояснение
dtype	Тип данных, или dict из имени столбца и типа данных. Тип данных, в который нужно преобразовать DataFrame. Это может быть один тип данных для всех столбцов или словарь, определяющий тип данных для каждого столбца
copy	bool, по умолчанию True. Возвращать ли новый DataFrame с указанными типами данных или модифицировать существующий DataFrame на месте
errors	{'raise', 'ignore'}, по умолчанию 'raise'. Определяет, как обрабатывать ошибки при преобразовании типов данных. Если 'raise', то при любой ошибке преобразования типа данных будет выдана ошибка. Если 'ignore', то ошибки будут проигнорированы, и будут возвращены исходные данные

Использование датасета

```
print("Изначальный DataFrame:")
print(df)
print("\nТипы данных:")
print(df.dtypes)

df['Age'] = df['Age'].astype(float)

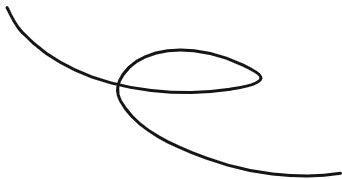
df['Rating'] = df['Rating'].astype(int)

print("\nDataFrame после изменения типов данных:")
print(df)
print("\nТипы данных после изменения:")
print(df.dtypes)
```



Нормализация текстовых данных —

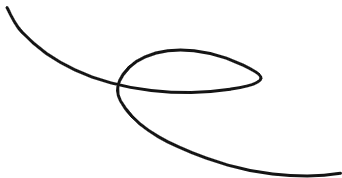
это этап предварительной обработки, который часто выполняется перед задачами обработки естественного языка (NLP): классификация текстов, анализ настроения или моделирование тем. Включает в себя преобразование текстовых данных к единообразному формату путем удаления строчных букв и специальных символов.





Перевод текстовых данных в нижний регистр —

это метод, переводящий в нижний регистр все символы в тексте. Обеспечивает согласованность текстовых данных и помогает снизить сложность задач обработки текста.



Перевод текстовых данных в нижний регистр



- Синтаксис: `DataFrame['текстовое_поле'].str.lower()`
- Возвращает: Серия с текстовыми данными, преобразованными в нижний регистр

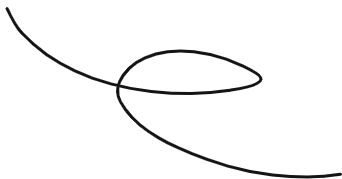
Пример перевода текстовых данных в нижний регистр

```
df['TextColumn'] = df['TextColumn'].str.lower()
```

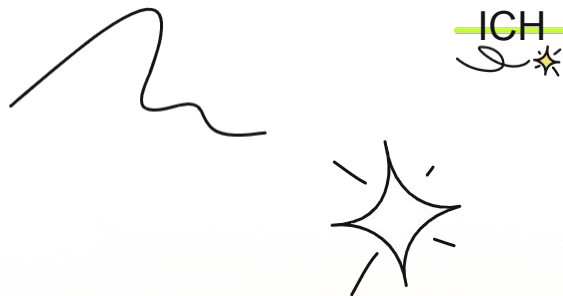



Удаление специальных символов —

это метод, включающий в себя удаление неалфавитно-цифровых символов, знаков препинания и других символов, которые могут быть не важны для задач анализа текста.



Удаление специальных символов



- Синтаксис: `DataFrame['текстовое_поле'].str.replace(r'^\w\s', '')`
- Возвращает: Серия, в которой из текстовых данных удалены специальные символы
- Параметр: `r'^\w\s'`: Шаблон регулярного выражения, который соответствует любому символу, не являющемуся символом слова или пробелом

Пример удаления специальных символов

```
df['TextColumn'] = df['TextColumn'].str.replace(r'[^\w\s]', '')
```

Использование датасета

```
import pandas as pd

df = pd.DataFrame({
    'TextColumn': ['Это образец ТЕКСТА со СПЕЦИАЛЬНЫМИ символами!',
                  'Еще один образец, с пунктуацией!!!!',
                  'Нормализация текстовых данных важна для задач НЛП']
})

df['TextColumn'] = df['TextColumn'].str.lower()

df['TextColumn'] = df['TextColumn'].str.replace(r'^\w\s', " ")

print(df)
```

Экспресс-опрос

?

Какими методами обрабатываются пропущенные значения?



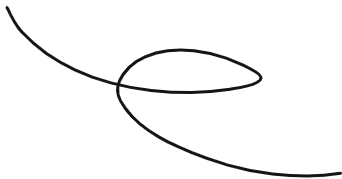
ВОПРОСЫ





Масштабирование числовых признаков —

это распространенный этап предварительной обработки в машинном обучении и анализе данных. Включает в себя преобразование числовых столбцов к общему масштабу, что может улучшить производительность определенных алгоритмов и моделей.



Использование scikit-learn для масштабирования

Min-Max Scaling (MinMaxScaler)

Standard Scaling (StandardScaler)

Эти техники изменяют масштаб числовых характеристик до определенного диапазона или распределения

Экспресс-опрос

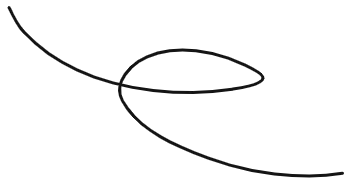
?

Что такое Min-Max Scaling?



Min-Max Scaling —

это техника масштабирования, которая масштабирует числовые характеристики к заданному диапазону, обычно от 0 до 1.



Масштабирование по Мин-Максу (MinMaxScaler)

- Синтаксис: `MinMaxScaler(feature_range=(min, max))`
- Параметры: `feature_range`: кортеж (min, max), по умолчанию=(0, 1). Желаемый диапазон преобразованных данных

Пример масштабирования по Мин-Максу

```
from sklearn.preprocessing import MinMaxScaler  
scaler = MinMaxScaler(feature_range=(0, 1))  
scaled_data = scaler.fit_transform(data)
```

Экспресс-опрос

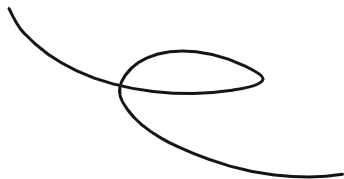
?

Что такое StandardScaler?



StandardScaler —

это техника масштабирования, которая масштабирует числовые признаки так, чтобы их среднее значение было равно 0, а стандартное отклонение - 1.



Стандартное масштабирование (StandardScaler)

- Синтаксис: `StandardScaler()`

Пример стандартного масштабирования

```
from sklearn.preprocessing import StandardScaler  
  
# Создайте объект StandardScaler  
scaler = StandardScaler()  
  
# Подгонка и преобразование данных  
scaled_data = scaler.fit_transform(data)
```

Использование датасета

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler, StandardScaler

# Образец DataFrame с числовыми характеристиками
data = pd.DataFrame({
    'Feature1': [10, 20, 30, 40, 50],
    'Feature2': [0.1, 0.2, 0.3, 0.4, 0.5]
})

# Min-Max Scaling
min_max_scaler = MinMaxScaler(feature_range=(0, 1))
scaled_data_min_max = min_max_scaler.fit_transform(data)
```


Использование датасета

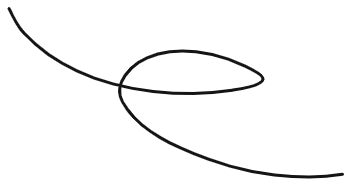
```
# Стандартное масштабирование
standard_scaler = StandardScaler()
scaled_data_standard = standard_scaler.fit_transform(data)

# Отображение масштабированных данных
print("Min-Max масштабированных данных:")
print(scaled_data_min_max)
print("\nСтандартные масштабированные данные:")
print(scaled_data_standard)
```



Выявление выбросов —

это шаг в анализе данных, позволяющий понять их распределение и обнаружить необычные или потенциально ошибочные наблюдения.



Экспресс-опрос

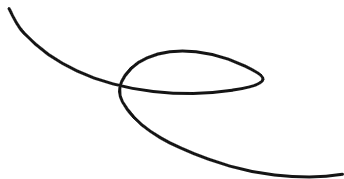
?

Что такое Boxplot?



Boxplot —

это метод, который визуализирует распределение числовых данных по квартилям, выделяя все точки данных, которые находятся за пределами усов (т. е. потенциальные выбросы).



Метод визуализации Boxplot

- Синтаксис: `DataFrame.boxplot(column='numeric_column', figsize=(width, height))`
- Возвращает: Объект Matplotlib Axes, представляющий график boxplot



Параметры метода визуализации Voxplot

Параметр	Пояснение
column	str или список str, по умолчанию None. Имя столбца или список имен столбцов для построения графика
figsize	Кортеж, по умолчанию (ширина, высота). Ширина и высота графика в дюймах

Пример метода визуализации Boxplot

```
# Визуализация выбросов с помощью боксплота  
df.boxplot(column='NumericColumn', figsize=(8, 6))
```



Диаграмма рассеяния —

это метод, который визуализирует связь между двумя числовыми переменными, позволяя заметить точки данных, которые значительно отклоняются от общей тенденции.

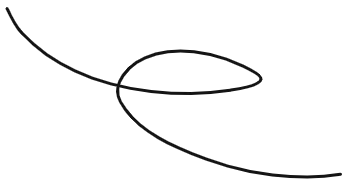


Диаграмма рассеяния



- Синтаксис: `DataFrame.plot.scatter(x='x_column', y='y_column', figsize=(width, height))`
- Возвращает: Объект Matplotlib Axes, представляющий диаграмму рассеяния

Параметры диаграмма рассеяния

Параметр	Пояснение
x: str	Имя столбца, используемого для оси x
y: str	Имя столбца для оси y
figsize	Кортеж, по умолчанию (ширина, высота). Ширина и высота графика в дюймах

Пример диаграммы рассеяния

```
# Визуализация выбросов с помощью диаграммы рассеяния  
df.plot.scatter(x='NumericColumn1', y='NumericColumn2', figsize=(8, 6))
```

Статистические методы

Метод z-score

Метод IQR (Interquartile Range)

Эти методы помогают количественно определить выбросы,
основываясь на их отклонении от среднего значения или медианы данных

Экспресс-опрос

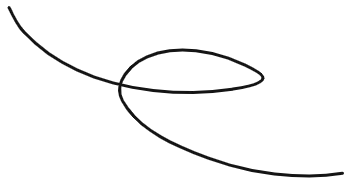


Что такое z-score?



z-score —

это метод, который измеряет, на сколько стандартных отклонений точка данных отклоняется от среднего значения набора данных. Точки данных с z-коэффициентом, превышающим определенный порог (например, 3 или -3), считаются выброшенными.



Метод z-score

- Синтаксис: `scipy.stats.zscore(data, axis=0)`
- Возвращает: Массив z-баллов для каждой точки данных



Параметры метода z-score

Параметр	Пояснение
data	Массив. Входные данные
axis	int или None, по умолчанию 0. Ось, вдоль которой вычисляются z-score. Если None, то вычисление производится по всему массиву

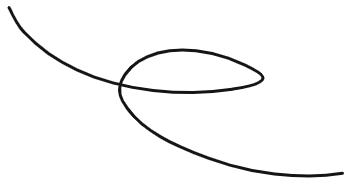
Пример метода z-score

```
import scipy.stats  
  
# Рассчитываем z-скор для 'NumericColumn' в DataFrame  
z_scores = scipy.stats.zscore(df['NumericColumn'])
```



IQR —

это межквартильный интервал, который измеряет разброс средних 50% данных. Выбросы обычно определяются как точки данных, которые находятся ниже $Q1 - 1,5 * IQR$ или выше $Q3 + 1,5 * IQR$.



Метод IQR

- Синтаксис: `DataFrame['column'].quantile(q)`
- Возвращает: Значение квантиля
- Параметры: `q`: float или массив типа float. Квантиль или последовательность квантилей для вычисления, в диапазоне от 0 до 1



Пример метода IQR

```
# Вычислить квартили для 'NumericColumn' в DataFrame  
Q1 = df['NumericColumn'].quantile(0.25)  
Q3 = df['NumericColumn'].quantile(0.75)  
IQR = Q3 - Q1
```

Использование набора данных примера

```
import pandas as pd
import matplotlib.pyplot as plt

# Образец DataFrame с числовыми характеристиками
df = pd.DataFrame({
    'NumericColumn': [10, 20, 30, 40, 50, 200],
    'NumericColumn1': [1, 2, 3, 4, 5, 100],
    'NumericColumn2': [0.1, 0.2, 0.3, 0.4, 0.5, 10]
})

# Визуализация выбросов с помощью бокс-графика
plt.figure(figsize=(8, 6))
df.boxplot(column='NumericColumn')
plt.title('Box Plot of NumericColumn')
plt.show()
```

Использование набора данных примера

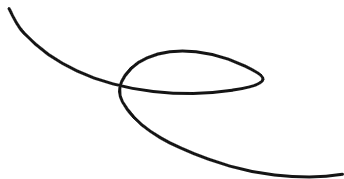
```
# Рассчитайте z-баллы для 'NumericColumn' в DataFrame
z_scores = scipy.stats.zscore(df['NumericColumn'])
print("Z-Scores для NumericColumn:")
print(z_scores)

# Рассчитайте квартили для 'NumericColumn' в DataFrame
Q1 = df['NumericColumn'].quantile(0.25)
Q3 = df['NumericColumn'].quantile(0.75)
IQR = Q3 - Q1
print("\nИнтерквартильный диапазон (IQR) для числового столбца:", IQR)
```



Обработка выбросов —

это важный этап предварительной обработки данных, позволяющий избежать их чрезмерного влияния на результаты анализа или моделирования.



Экспресс-опрос

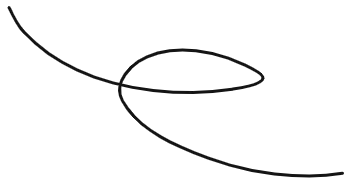
?

Какие статистические методы используются для выявления выбросов?



Удаление выбросов —

это удаление данных, которые считаются выбросами на основе определенных критериев, таких как превышение определенного порога по отношению к среднему значению или медиане данных.



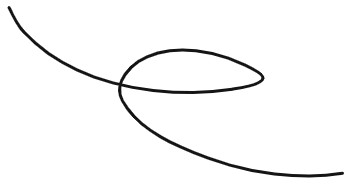
Пример удаления выбросов

```
# Удаление выбросов столбце 'NumericColumn' на основе z-score  
df_no_outliers = df[(df['NumericColumn'] > mean - 3 * std) & (df['NumericColumn'] < mean + 3 * std)]
```



Выделение выбросов —

это замена значений выбросов заранее определенным пороговым значением, чтобы ограничить их влияние на данные. Выбросы, превышающие порог, устанавливаются на пороговое значение.



Выделение выбросов



- Синтаксис: `DataFrame['column'] = DataFrame['column'].clip(lower=lower_bound, upper=upper_bound)`
- Возвращает: DataFrame с заблокированными значениями выбросов

Параметры выделения выбросов

Параметр	Пояснение
lower_bound	float или int, необязательно. Нижняя граница порога для обрезания выбросов
upper_bound	float или int, необязательно. Верхняя граница порога для выделения выбросов

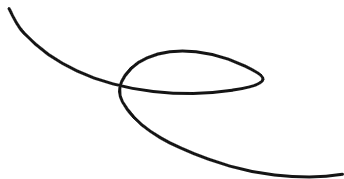
Пример выделения выбросов

```
# Отсечь выбросы в 'NumericColumn' на 99-м процентиле  
upper_bound = df['NumericColumn'].quantile(0.99)  
df['NumericColumn'] = df['NumericColumn'].clip(upper=upper_bound)
```



Преобразование выбросов —

это применение математических преобразований к данным, чтобы уменьшить влияние выбросов, сохранив при этом общее распределение.



Преобразование выбросов



- Синтаксис: `DataFrame['column'] = func(DataFrame['column'])`
- Возвращает: `DataFrame` с преобразованными значениями выбросов
- Параметры: `func`: функция. Функция преобразования, применяемая к столбцу данных.

Пример преобразования выбросов

```
import numpy as np

# Лог-трансформируем выбросы в 'Числовом столбце'
df['NumericColumn'] = np.log1p(df['NumericColumn'])
```

Использование набора данных примера

```
import pandas as pd

# Образец DataFrame с числовыми признаками
df = pd.DataFrame({
    'NumericColumn': [10, 20, 30, 40, 50, 200]
})

# Удаляем выбросы в 'NumericColumn' на основе порогового значения
mean = df['NumericColumn'].mean()
std = df['NumericColumn'].std()
df_no_outliers = df[(df['NumericColumn'] > mean - 3 * std) & (df['NumericColumn'] < mean + 3 * std)]

# Отметим выбросы в 'NumericColumn' на 99-м процентиле
upper_bound = df['NumericColumn'].quantile(0.99)
df_capped = df.copy()
df_capped['NumericColumn'] = df_capped['NumericColumn'].clip(upper=upper_bound)
```

Использование набора данных примера

```
# Преобразование выбросов в 'NumericColumn' с помощью лог-преобразования
import numpy as np
df_transformed = df.copy()
df_transformed['NumericColumn'] = np.log1p(df_transformed['NumericColumn'])

# Отображение исходного и преобразованного фреймов данных
print("Изначальный DataFrame:")
print(df)
print("\nDataFrame после удаления выбросов:")
print(df_no_outliers)
print("\nDataFrame после обрезания выбросов:")
print(df_capped)
print("\nDataFrame после преобразования выбросов:")
print(df_transformed)
```

Экспресс-опрос

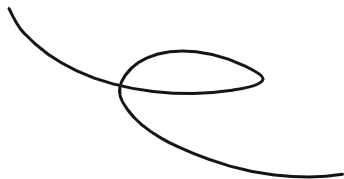
?

Что такое категориальные данные?



Обработка категориальных данных —

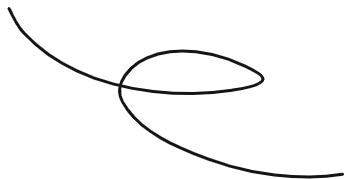
это важная задача при предварительной обработке данных, особенно в машинном обучении и статистическом анализе.





Категориальные данные —

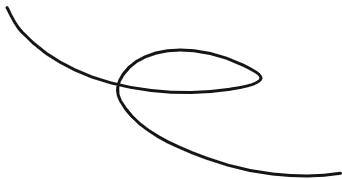
это дискретные значения, которые принадлежат к определенной категории или классу и могут не иметь числового значения.





Использование библиотеки pandas —

предлагает удобные методы для работы с категориальными данными в объектах DataFrame. Она позволяет эффективно преобразовывать, манипулировать и анализировать категориальные переменные.



Кодирование категориальных данных



- Синтаксис: `encoded_data = pd.get_dummies(data, columns=['categorical_column'])`

Параметры кодирования категориальных данных

Параметр	Пояснение
data	DataFrame, содержащий категориальный(ые) столбец(ы), подлежащий(ие) кодированию
columns	Список имен столбцов или индексных позиций категориальных столбцов для кодирования

Пример кодирования категориальных данных

```
import pandas as pd

# Создаем DataFrame с категориальными данными
data = pd.DataFrame({'Category': ['A', 'B', 'A', 'C', 'B']})

# Кодируем категориальные данные с помощью одноточечного кодирования
encoded_data = pd.get_dummies(data, columns=['Category'])
print("Кодированные данные:")
print(encoded_data)
```

Кодировка лейблов



- Синтаксис: `data['encoded_column'] = pd.factorize(data['categorical_column'])[0]`
- Параметры: `categorical_column`: Имя или индекс категориального столбца для кодирования

Пример кодировки лейблов

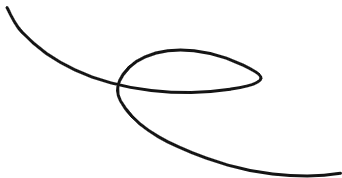
```
import pandas as pd

data['Encoded_Category'] = pd.factorize(data['Category'])[0]
print("Лейбл-кодированные данные:")
print(data)
```



Использование библиотеки scikit-learn —

предоставляет дополнительную функциональность для работы с категориальными данными, включая методы кодирования, преобразования и извлечения признаков.



Пример ординального кодирования

```
from sklearn.preprocessing import OrdinalEncoder

# Инициализация объекта OrdinalEncoder
encoder = OrdinalEncoder()

# Подгонка и преобразование категориальных данных
encoded_data = encoder.fit_transform(data[['Category']])
print("Ординарные кодированные данные:")
print(encoded_data)
```

Стать IT-шником может каждый



OneHotкодирование



- Синтаксис: `from sklearn.preprocessing import OneHotEncoder`
- Параметры: `drop`: Нужно ли отбросить одну из категорий, чтобы избежать мультиколлинеарности.

Пример OneHotкодирования

```
from sklearn.preprocessing import OneHotEncoder

# Инициализация объекта OneHotEncoder
encoder = OneHotEncoder(drop='first')

# Подгонка и преобразование категориальных данных
encoded_data = encoder.fit_transform(data[['Category']]).toarray()
print("One-Hot кодированные данные:")
print(encoded_data)
```


Использование набора данных примера

```
import pandas as pd
from sklearn.preprocessing import OneHotEncoder

# Образец DataFrame с категориальными данными
data = pd.DataFrame({'Category': ['A', 'B', 'A', 'C', 'B']})

# Одномоментное кодирование категориальных данных с помощью pandas
encoded_data_pandas = pd.get_dummies(data, columns=['Category'])
print("One-Hot кодированные данные (pandas):")
print(encoded_data_pandas)

# One-hot кодирование категориальных данных с помощью scikit-learn
encoder = OneHotEncoder(drop='first')
encoded_data_sklearn = encoder.fit_transform(data[['Category']]).toarray()
print("\nOne-Hot кодированные данные (scikit-learn):")
print(encoded_data_sklearn)
```



ВОПРОСЫ





Набор данных Wine Recognition

Это набор данных для распознавания вин, доступный в библиотеке `scikit-learn`, является классическим эталонным набором данных, обычно используемым для задач классификации.

Набор данных Wine Recognition



Этот набор данных включает результаты химического анализа вин, выращенных в одном и том же регионе Италии, но полученных из трех разных сортов (классов)

Набор данных Wine Recognition



Этот набор данных включает результаты химического анализа вин, выращенных в одном и том же регионе Италии, но полученных из трех разных сортов (классов)

В нем содержится 13 числовых признаков, представляющих различные химические свойства, обнаруженные в винах



Классы

Это целевая переменная. Существует три класса сортов вина, представленных целыми числами.

Классы сортов вина

Класс 0

Класс 1

Класс 2

Вино сорта 1

Вино сорта 2

Сорт вина 3



Характеристики

Это 13 признаков (атрибутов), представляющих различные химические свойства, обнаруженные в винах.

Характеристики



Алкоголь



Яблочная кислота



Зола



Щелочность золы



Магний



Общие фенолы



Флаваноиды



Нефлаваноидные фенолы



Проантоцианы



Интенсивность цвета



Оттенок



OD280/OD315 разбавленных вин



Пролин



Размер данных

Это 178 экземпляров (строк), каждый из которых представляет собой отдельный образец вина.



Источник данных

Это репозиторий машинного обучения
Калифорнийского университета в Ирвине (UCI).



Использование

Для задач классификации, в частности для обучения и оценки моделей машинного обучения.

Установка необходимых библиотек

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_wine
import matplotlib.pyplot as plt
import seaborn as sns
```

Загрузка данных

```
X, y = load_wine(return_X_y=True, as_frame=True)
X['class'] = y
X.head()
```

Повторный вызов функции load_wine()

```
print(load_wine()['DESCR'])
```

Экспресс-опрос

?

Что такое `.describe()`?



.describe()

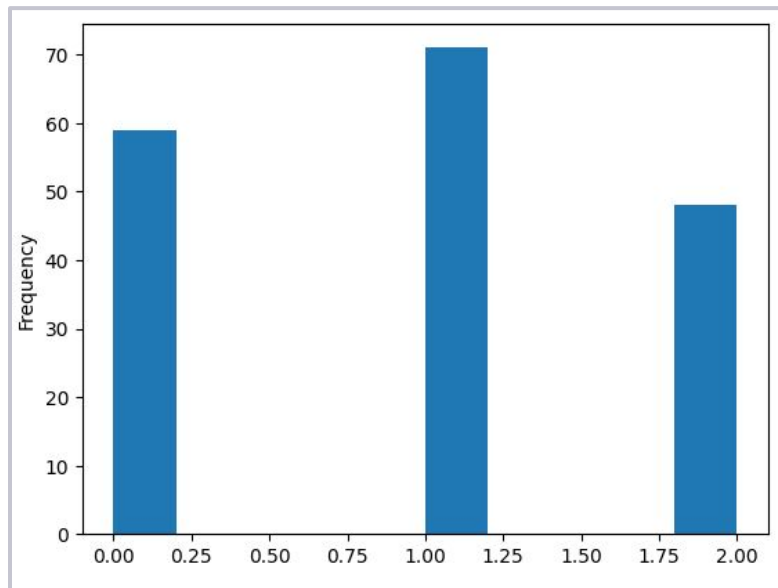
Это метод, который вычисляет и отображает описательную статистику для каждого числового столбца в DataFrame X.

Метод .describe()

```
X.describe()
```

Распределение классов

```
X['class'].plot.hist()
```



Экспресс-опрос

?

Что такое `.plot.hist()`?



`.plot.hist()`

Это метод, который строит гистограмму для данных в выбранном столбце (`X['class']`).



Гистограмма

Это графическое представление, которое отображает распределение числовых данных путем разделения данных на бины или интервалы и подсчета количества наблюдений, попавших в каждую бину.

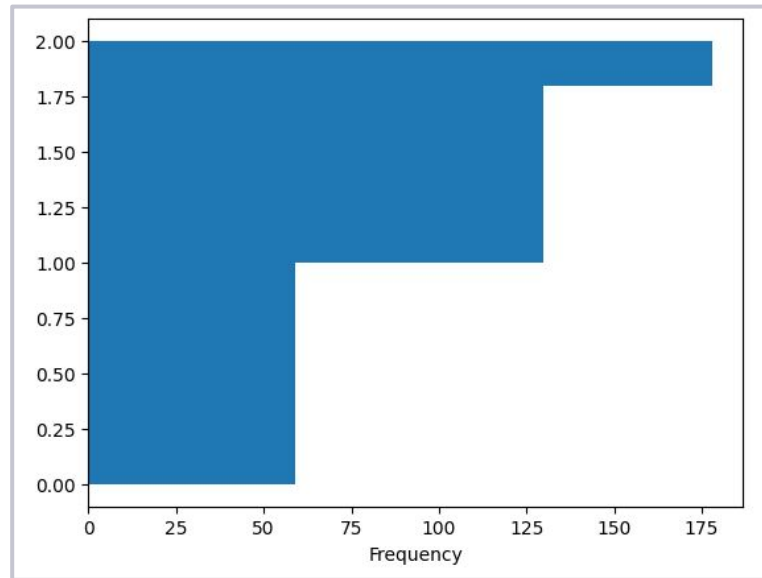


`.hist()`

Это функция, которая используется для создания графика гистограммы в pandas.

Создание гистограммы

```
X['class'].plot.hist(orientation="horizontal", cumulative=True)
```

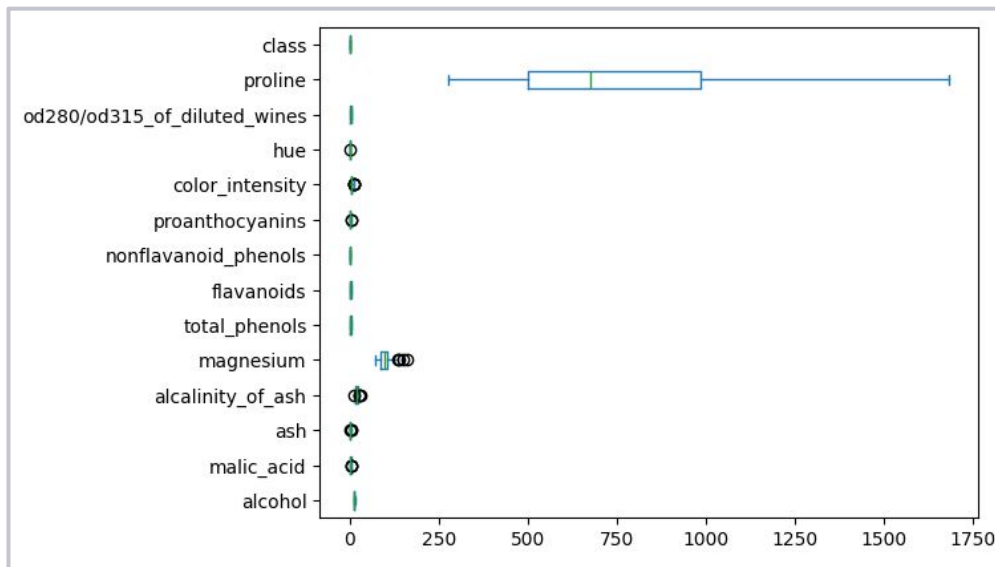


Вспомним Python Fundamentals

```
from collections import Counter  
  
print(Counter(y))
```

Горизонтальный бокс-график

`X.plot.box(vert=False)`



Экспресс-опрос

A purple square icon containing a white question mark.

Что такое `.plot.box()`?



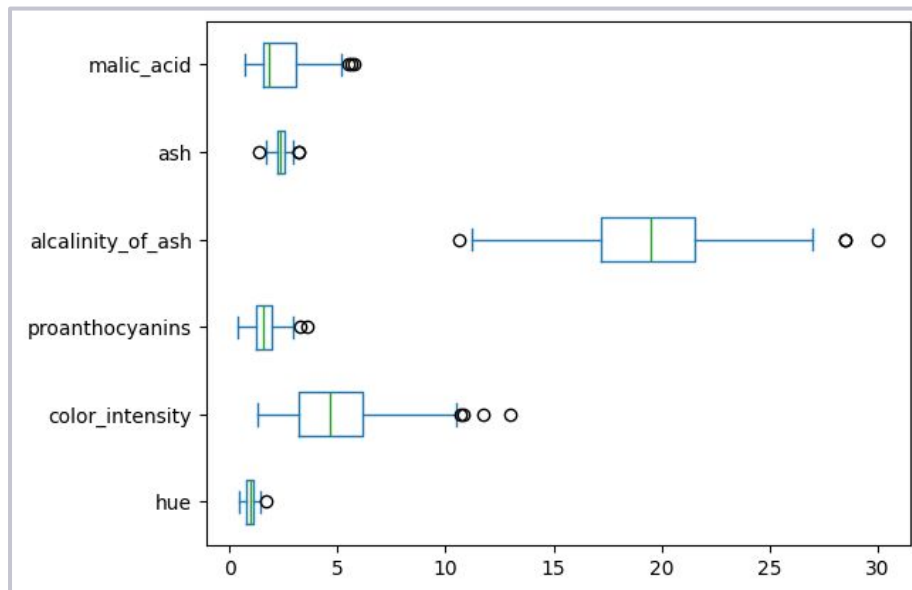
`.plot.box()`

Это метод, который генерирует квадратный график для данных в DataFrame X.

Выбросы

```
with_outliners = ['hue', 'color_intensity', 'proanthocyanins',  
                  'alcalinity_of_ash', 'ash', 'malic_acid'] # , 'magnesium'  
X[with_outliners].plot.box(vert=False)
```

Выбросы



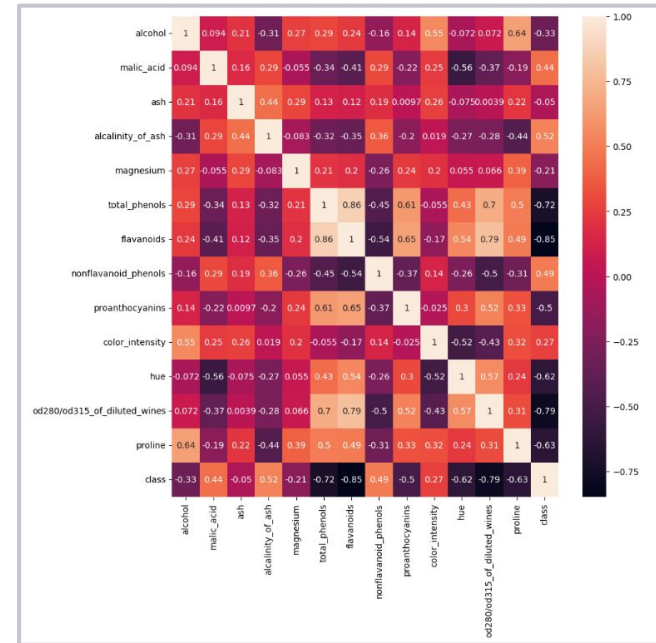
Экспресс-опрос

?

Что бы Вы сделали с выбросами?

Корреляция и тепловая карта

```
plt.figure(figsize=(10,10))
cmap=X.corr()
sns.heatmap(cmap, annot=True)
```



Экспресс-опрос

?

Что такое plt.figure()?



`plt.figure()`

Это функция, которая создает новую фигуру с заданным размером в Matplotlib. Она задает размер фигуры, чтобы визуализация тепловой карты имела достаточно места для четкого отображения.



Корреляционная матрица

Это квадратная матрица, каждый элемент которой представляет собой коэффициент корреляции между двумя признаками. Она дает представление о связях и зависимостях между различными признаками в наборе данных.

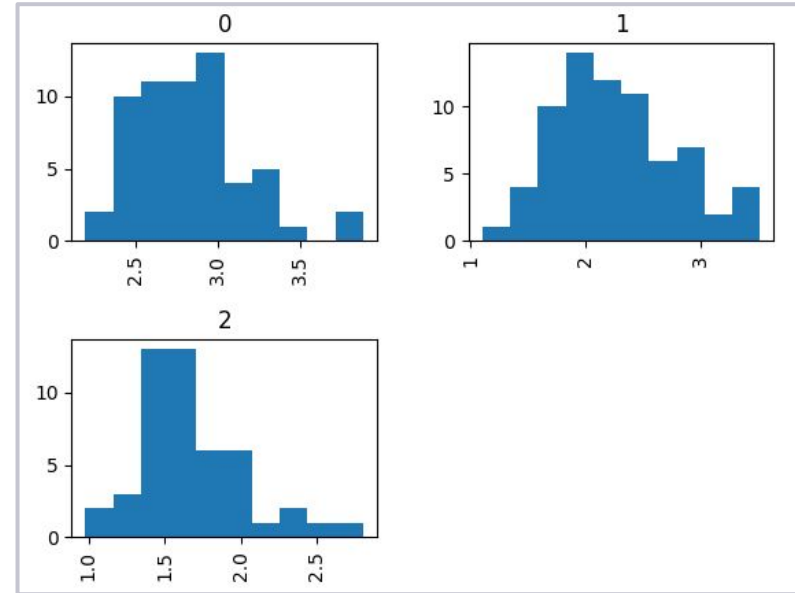


heatmap()

Это функция, которая генерирует тепловую карту. Она визуализирует корреляционную матрицу (сmap) в виде тепловой карты, где цвет каждой ячейки представляет собой коэффициент корреляции между двумя признаками.

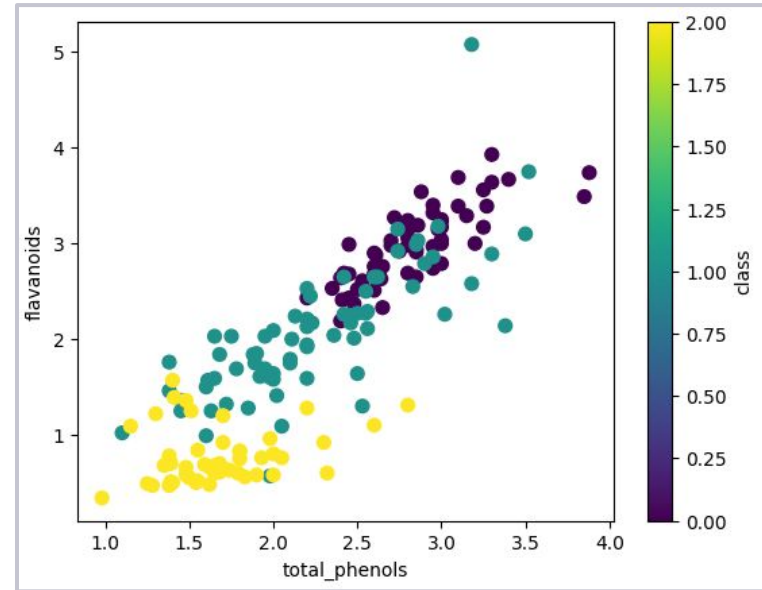
Корреляция и тепловая карта

```
X['total_phenols'].hist(by=X['class'])
```



Корреляция и тепловая карта

```
X.plot.scatter(x="total_phenols",  
               y="flavanoids", c="class",  
               s=50, colormap='viridis')
```



Графики рассеяния для атрибутов с лучшей корреляцией

```
good_corr = ['total_phenols', 'flavanoids', 'od280/od315_of_diluted_wines', 'hue', 'proline',
'nonflavanoid_phenols', 'alcalinity_of_ash']

pairs = [(good_corr[i], good_corr[j]) for i in range(len(good_corr)) for j in range(i+1,
len(good_corr))]

non_repeated_pairs = []
seen_pairs = set()
for pair in pairs:
    reversed_pair = (pair[1], pair[0]) # Check both (a, b) and (b, a)
    if pair not in seen_pairs and reversed_pair not in seen_pairs:
        non_repeated_pairs.append(pair)
        seen_pairs.add(pair)

print(len(non_repeated_pairs))
```

Графики рассеяния для атрибутов с лучшей корреляцией

```
num_plots = len(non_repeated_pairs)

num_rows = num_plots // 3
num_cols = 3
fig, axs = plt.subplots(num_rows, num_cols, figsize=(24, 24))

if num_rows == 1 or num_cols == 1:
    axs = axs.flatten()

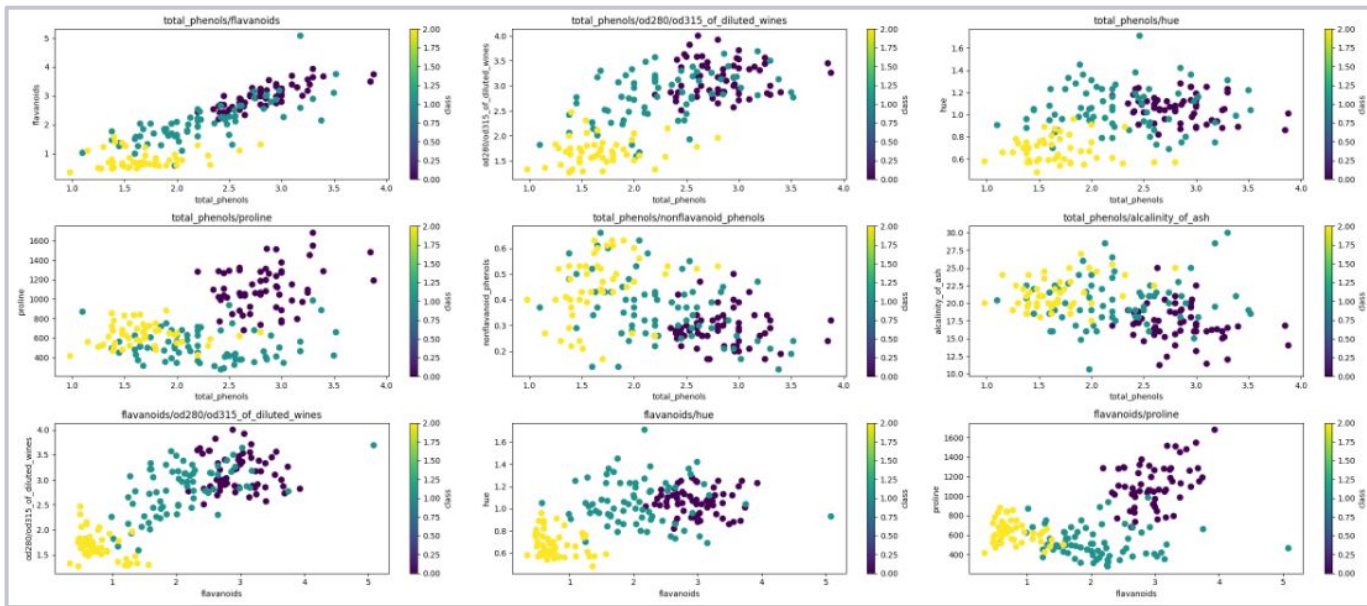
axe = axs.ravel()

for i, pair in enumerate(non_repeated_pairs):
    X.plot.scatter(x=pair[0], y=pair[1], c="class", s=50, colormap='viridis', ax=axe[i])
    axe[i].set_title(f'{pair[0]}'+ '/' +f'{pair[1]}')

plt.tight_layout()

plt.show()
```


Графики рассеяния для атрибутов с лучшей корреляцией



Экспресс-опрос

?

Как сделать гистограмму вертикальной?



ВОПРОСЫ





GeoPandas

Это проект с открытым исходным кодом, призванный упростить работу с геопространственными данными на python. Расширяет типы данных, используемые pandas, чтобы позволить пространственные операции над геометрическими типами.

Установка GeoPandas

1. GeoPandas совместим с Python 3.6 или выше.
2. Установите зависимости: GeoPandas зависит от нескольких других библиотек, включая pandas, Shapely, Fiona, Matplotlib и PyProj. Вы можете установить эти зависимости с помощью менеджера пакетов, например pip.



Подсказка:

```
pip install pandas shapely
fiona matplotlib pyproj
```

Установка GeoPandas

1. GeoPandas совместим с Python 3.6 или выше.
2. Установите зависимости: GeoPandas зависит от нескольких других библиотек, включая pandas, Shapely, Fiona, Matplotlib и PyProj. Вы можете установить эти зависимости с помощью менеджера пакетов, например pip.
3. Установите GeoPandas: После установки зависимостей вы можете установить GeoPandas с помощью pip.



Подсказка:

```
pip install pandas shapely
fiona matplotlib pyproj
```

```
pip install geopandas
```

Установка GeoPandas

1. GeoPandas совместим с Python 3.6 или выше.
2. Установите зависимости: GeoPandas зависит от нескольких других библиотек, включая pandas, Shapely, Fiona, Matplotlib и PyProj. Вы можете установить эти зависимости с помощью менеджера пакетов, например pip.
3. Установите GeoPandas: После установки зависимостей вы можете установить GeoPandas с помощью pip.
4. Проверьте установку: После установки вы можете проверить правильность установки GeoPandas, импортировав его в сценарий Python или интерактивную сессию.



Подсказка:

```
pip install pandas shapely
fiona matplotlib pyproj
```

```
pip install geopandas
```

```
import geopandas as gpd
```

GeoPandas



Не требует никаких дополнительных шагов по настройке.
Можно сразу же начать использовать его для работы с геопространственными данными

Экспресс-опрос

?

Какие атрибуты геометрий есть в GeoPandas?



GeoDataFrame

Это фундаментальная структура данных в GeoPandas, расширяющая возможности pandas DataFrame для работы с пространственными данными. Представляет собой табличные данные с колонкой геометрии, в которой хранятся геометрические объекты, представляющие пространственные особенности, связанные с каждой строкой.

Ключевые аспекты GeoDataFrame



Табличные данные с геометрическим столбцом



Интеграция с Shapely



Пространственные атрибуты и методы



Обработка CRS



Атрибуты и пространственные запросы



Построение графиков и визуализация

Пример

```
import geopandas as gpd
from shapely.geometry import Point

data = {
    'City': ['New York', 'Los Angeles', 'Chicago', 'Houston'],
    'Population': [8419600, 3990456, 2705994, 2325502]
}
df = pd.DataFrame(data)

geometry = [Point(-74.006, 40.7128), # New York (longitude, latitude)
            Point(-118.2437, 34.0522), # Los Angeles
            Point(-87.6298, 41.8781), # Chicago
            Point(-95.3698, 29.7604)] # Houston

gdf = gpd.GeoDataFrame(df, geometry=geometry, crs='EPSG:4326')

print(gdf)
```

Использование geodatasets



В GeoPandas данные можно подгрузить из внешних источников, например, по url

Работа с данными из библиотеки geodatasets

```
import geopandas
from geodatasets import get_path

path_to_data = get_path("geoda airbnb")
gdf = geopandas.read_file(path_to_data)

gdf = gdf.set_index('community')
gdf.head()
```

Характеристики геометрий



В наборе данных Airbnb есть только один столбец геометрии, этот столбец автоматически становится активной геометрией



GeoDataFrame.area

Это атрибут, который позволяет измерить площадь каждого полигона и возвращает `pandas.Series`. Это просто `GeoSeries.area`, примененная к активному столбцу геометрии.

Измерение площади

```
gdf['area'] = gdf.area
gdf['area']
```



GeoDataFrame.boundary

Это атрибут, который позволяет получить границу каждого полигона (LineString).

Получение границ полигонов и центроида

```
gdf['boundary'] = gdf.boundary  
gdf['boundary']
```

Создание новых геометрий

```
gdf['centroid'] = gdf.centroid  
gdf['centroid']
```

Измерение расстояния

```
first_point = gdf['centroid'].iloc[0]  
gdf['distance'] = gdf['distance'].distance(first_point)  
gdf['distance']
```



Построение карт

Позволяет проверить, как геометрия выглядит в пространстве.



GeoDataFrame.plot()

Это функция, которая позволяет построить активную геометрию.

Построение карт



Чтобы сделать цветовое кодирование по другому столбцу,
необходимо передать этот столбец в качестве первого аргумента

График по столбцу активной геометрии

```
gdf.plot('area', legend=True)
```



GeoDataFrame.explore()

Это функция, которая позволяет исследовать данные в интерактивном режиме. Ведет себя так же, как и `plot()`, но вместо нее возвращает интерактивную карту.

GeoDataFrame.explore()

```
gdf.explore('area', legend=False)
```

GeoDataFrame.set_geometry

```
gdf = gdf.set_geometry('centroid')
gdf.plot('area', legend=True)
```

Наложение GeoSeries

```
ax = gdf['geometry'].plot()
gdf['centroid'].plot(ax=ax, color='black')
```

Экспресс-опрос

?

Как можно наложить два графика друг на друга?



ВОПРОСЫ





ЗАДАНИЕ



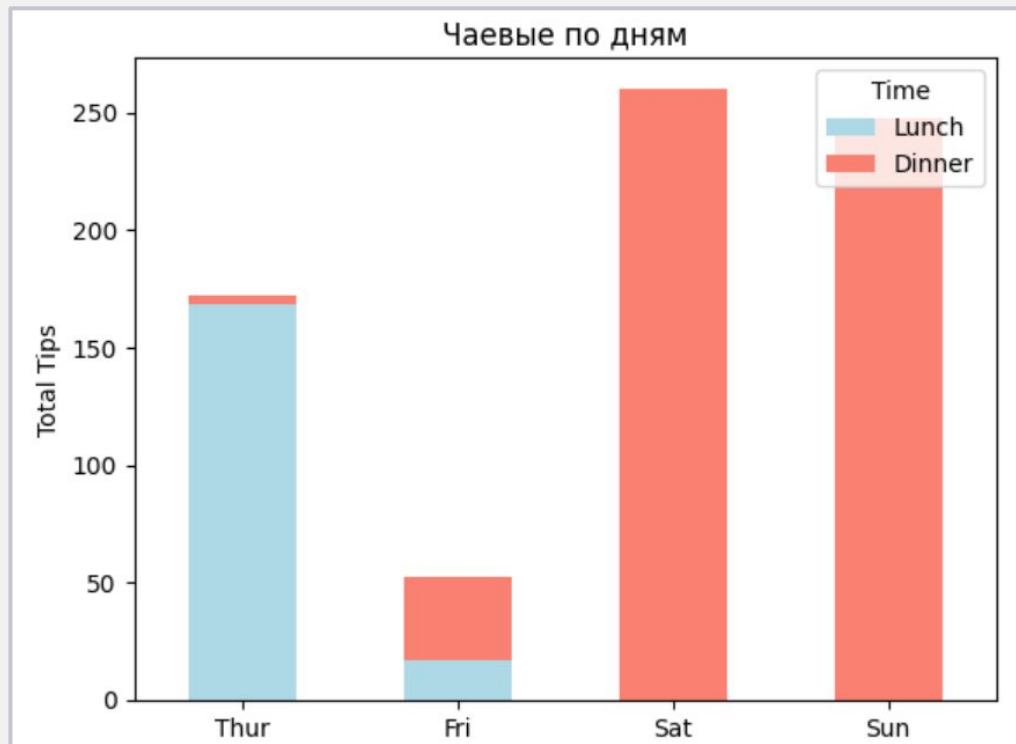
Задание

Завершить работу над задачами с практикумов 9 и 10.



1. Датасет iris: Визуализируйте числовую характеристику с помощью квадратной диаграммы, чтобы выявить выбросы. Используйте статистические методы (например, IQR) для программного обнаружения выбросов. Выберите один из методов (удаление, усечение или преобразование) для работы с промахами. Реализуйте выбранный метод и проверьте результаты.
2. Создайте DataFrame со столбцом 'Height', представляющим рост студентов. Внесите выбросы в столбец 'Height'. Реализуйте метод для обнаружения и обработки выбросов.
3. Создайте DataFrame со столбцами 'Age' и 'Salary'. Масштабируйте столбцы 'Age' и 'Salary' с помощью масштабирования Min-Max.

1. Повторите следующий график, основываясь на данных из датасета tips:





Задание

2. Постройте столбчатую диаграмму, сравнивая продолжительность жизни со скоростью животного. Сделайте диаграмму горизонтальной (barh).
3. Загрузите набор данных titanic.
 - Подсчитайте количество пассажиров в каждом классе (столбец class).
 - Визуализируйте распределение пассажиров по классам с помощью круговой диаграммы.



ВОПРОСЫ





РАЗБОР ДОМАШНЕГО ЗАДАНИЯ



Домашнее задание

1. Загрузите датасет `titanic` из библиотеки `seaborn`, определите столбцы с пустыми значениями и заполните их любым из возможных методов.
2. Загрузите датасет `titanic` из библиотеки `seaborn`, определите категориальную переменную и закодируйте её.

Решение

```
1. import seaborn as sns

titanic_df = sns.load_dataset('titanic')

columns_with_missing = titanic_df.columns[titanic_df.isnull().any()].tolist()

titanic_df.fillna(method='ffill', inplace=True)
```


Решение

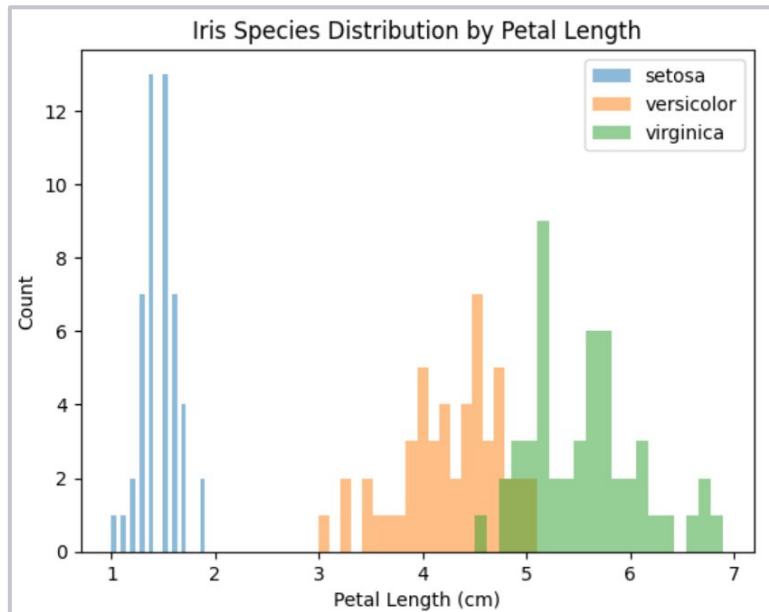
```
2.  from sklearn.datasets import load_iris

    iris = load_iris()
    iris_df = pd.DataFrame(data=iris.data, columns=iris.feature_names)
    iris_df['species'] = iris.target

    iris_df = pd.get_dummies(iris_df, columns=['species'])
```

Домашнее задание

Основываясь на датасете iris из seaborn, постройте следующий график:



Решение

```
import seaborn as sns
import matplotlib.pyplot as plt

iris_df = sns.load_dataset('iris')

iris_df.groupby('species')['petal_length'].plot(kind='hist', alpha=0.5, bins=20,
legend=True)
plt.title('Iris Species Distribution by Petal Length')
plt.xlabel('Petal Length (cm)')
plt.ylabel('Count')
plt.show()
```



ВОПРОСЫ



Заключение

